

Yes. One important clarification first: **“backward compatibility” is not a property that a feature either universally has or lacks.** Java compatibility has several dimensions:

- **Source compatibility** — old Java source can still compile on a newer JDK.
- **Binary compatibility** — old .class files can still link/run on a newer JVM.
- **Behavioral compatibility** — old programs continue to behave essentially as before.
- **Forward compatibility** — an **older** JDK can run/compile code produced for a **newer** JDK. Java generally does **not** guarantee this.

Oracle explicitly distinguishes source, binary, and behavioral compatibility, and the Java SE 26 JVM supports class-file versions **45 through 70**, while a Java 26-generated class file is version 70. ([Oracle Docs](#))

So, for example:

Java 26 JVM → Java 8 .class: generally supported

Java 8 JVM → Java 26 .class: not supported

Java 26 → Java 1: Features and Backward Compatibility

Java Version	Year	Major Feature(s)	Definition	Example	Older Java Source on Newer JDK?	Newer Feature on Older JDK?	Compatibility Notes
Java 26	2026	Primitive types in patterns, instanceof, switch (<i>preview</i>)	Allows primitive types to participate in pattern matching and related constructs.	if (x instanceof int i) { ... } (<i>preview</i>)	Yes, generally	No	Preview features require the corresponding preview-enabled compiler/runtime; Java 25 preview evolved into Java 26's fourth preview. (Oracle Docs)

Java 25	2025	Module Import Declarations; Compact Source Files; Flexible Constructor Bodies	Simplifies imports, small Java programs, and constructor initialization.	import module java.sql;	Yes, generally	No	These became permanent in Java 25. (Oracle Docs)
Java 24	2025	Module Import Statements; Primitive Patterns; Flexible Constructor Bodies; Simple Source Files (<i>previews</i>)	Experimental language improvements that preceded later permanent features.	Simplified source/class declarations	Yes	No	Preview functionality is not something an older JDK can understand.
Java 23	2024	Module Import Declarations; Primitive Patterns; Implicitly Declared Classes (<i>previews</i>)	Allows simpler source syntax and experimental pattern capabilities.	Simplified main program	Yes	No	Preview features are version-specific.
Java 22	2024	Unnamed Variables & Patterns; String Templates (<i>preview</i>); Statements Before super() (<i>preview</i>)	Allows intentionally unused variables/pattern components and experiments with simpler string construction.	if (obj instanceof String _) { ... }	Yes	No	Unnamed variables/patterns became permanent in Java 22; other listed features were preview. (Oracle Docs)
Java 21	2023	Virtual Threads; Sequenced Collections; Record Patterns; Pattern Matching for switch; Generational ZGC	Major modern Java release with lightweight concurrency and advanced pattern matching.	if (obj instanceof Point(int x, int y))	Yes, generally	No	Record patterns and pattern matching for switch became permanent. (Oracle Docs)

Java 20	2023	Record Patterns (<i>preview</i>); Pattern Matching for switch (<i>preview</i>)	Experimental versions of features later standardized.	case Point(int x, int y) -> ...	Yes	No	Preview features cannot be assumed to work on older releases.
Java 19	2022	Virtual Threads (<i>preview</i>); Pattern Matching for switch (<i>preview</i>); Structured Concurrency (<i>incubator</i>)	Introduced lightweight threads and structured concurrency concepts.	Thread.startVirtualThread(task);	Yes	No	Virtual threads became permanent in Java 21.
Java 18	2022	UTF-8 by Default; Simple Web Server; Code Snippets	Makes UTF-8 the default charset in standard situations and adds developer conveniences.	Charset.defaultCharset() → UTF-8 in standard cases	Yes	No	Existing Java programs may experience behavioral changes if they relied on a platform-specific default charset.
Java 17	2021	Sealed Classes; Pattern Matching improvements; Strong encapsulation	Sealed classes restrict which classes can extend/implement a type.	sealed class Shape permits Circle, Square {}	Yes, generally	No	Sealed classes became permanent. Strong encapsulation can break applications using JDK internals.
Java 16	2021	Records; Pattern Matching for instanceof; jpackage	Records provide compact immutable data carriers.	record Person(String name, int age) {}	Yes	No	Records and pattern matching became permanent in Java 16.
Java 15	2020	Text Blocks; Sealed Classes (<i>preview</i>); Hidden Classes	Text blocks provide convenient multiline strings.	"""Hello\nJava" """	Yes	No	Text blocks became permanent; sealed classes were still preview.

Java 14	2020	Switch Expressions; Pattern Matching instanceof (<i>preview</i>); Records (<i>preview</i>)	switch can produce a value.	int n = switch(x) { case 1 -> 10; default -> 0; };	Yes	No	Switch expressions became permanent.
Java 13	2019	Text Blocks (<i>preview</i>); switch Expressions (<i>second preview</i>)	Experimental multiline strings and expression-oriented switch.	String s = """"Hello"""";	Yes	No	Preview syntax is release-dependent.
Java 12	2019	Switch Expressions (<i>preview</i>); JVM Constants API	Begins expression-oriented switch evolution.	var result = switch(day) { ... };	Yes	No	Preview feature cannot be used directly by older JDK compilers.
Java 11	2018	var in Lambda Parameters; HTTP Client; Flight Recorder	Extends local-variable syntax and introduces modern HTTP client functionality.	(var x, var y) -> x + y	Yes	No	Java 11 removed/deprecated several old components; JDK 8 cannot compile Java 11 syntax/API.
Java 10	2018	Local Variable Type Inference (var)	Compiler infers local variable type while retaining static typing.	var name = "Anurag";	Yes	No	Java 9/8 cannot compile var syntax.
Java 9	2017	JPMS/Modules; jshell; private interface methods	Modules provide explicit dependencies/encapsulation.	module com.example.app { requires java.sql; }	Mostly	No	Major compatibility boundary. Internal JDK APIs became more strongly encapsulated; some old applications require migration. Oracle notes significant changes beginning with JDK 9. (Oracle Docs)

Java 8	2014	Lambda; Streams; Default Methods; Functional Interfaces; Optional; new Date/Time API	Introduced functional programming capabilities and stream processing.	<code>list.stream().filter(x -> x > 10).forEach(System.out::println);</code>	Yes	No	Java 7 cannot compile lambdas/streams. Java 8 retained substantial source/binary compatibility with earlier Java. (Oracle Docs)
Java 7	2011	Diamond; try-with-resources; multi-catch; String switch; binary literals	Simplifies generics, resource management, exceptions and literals.	<code>try (FileInputStream in = ...) {}</code>	Yes	No	Java 6 cannot compile Java 7 syntax. Oracle documents these language enhancements explicitly. (Oracle Docs)
Java 6	2006	Scripting API; Compiler API; Desktop API; concurrency/API enhancements	Adds APIs rather than major Java-language syntax changes.	<code>new ScriptEngineManager()</code>	Yes	No	No Java-language changes were introduced in Java SE 6, although many APIs were added/enhanced. (Oracle Docs)
Java 5 / 1.5	2004	Generics; Annotations; Enums; Autoboxing; Varargs; Enhanced for; Static import	Major language modernization adding type-safe generics, metadata and convenient syntax.	<code>List<String> names = new ArrayList<>();</code>	Yes	No	Java 1.4 compiler cannot compile generics, annotations, enums, etc.
Java 1.4	2002	Assertions; Regular Expressions; NIO; Logging; XML/JAXP; Preferences	Added assertion syntax and major library infrastructure.	<code>assert age >= 18;</code>	Yes	No	Java 1.3 cannot compile assert syntax.
Java 1.3	2000	HotSpot JVM; JNDI; Java Sound;	Improved JVM performance and	<code>Context ctx = ...;</code>	Yes	No	Mostly API/runtime evolution rather than

		performance improvements	enterprise/platform APIs.				major language syntax changes.
Java 1.2	1998	Collections Framework; Swing; strictfp; Java 2 platform	Introduced modern collections and Swing GUI framework.	List<String> list = new ArrayList<>();	Yes	No	New APIs are unavailable on Java 1.1 unless separately supplied.
Java 1.1	1997	Inner Classes; JDBC; JavaBeans; RMI; Reflection; Serialization	Added nested classes and major application/integration APIs.	class Outer { class Inner {} }	Yes	No	Java 1.0 cannot use 1.1 APIs/language features.
Java 1.0	1996	Classes; Objects; Inheritance; Interfaces; Exceptions; Threads; AWT; I/O; Networking	Initial public Java platform and object-oriented language.	class Hello { public static void main(String[] a) {} }	Baseline	No	There is no earlier Java SE release against which Java 1.0 must be backward-compatible.

The Java 9–26 language evolution is documented by Oracle, including permanent and preview features. ([Oracle Docs](#)) The official JVM specification lists class-file compatibility from Java 1.0.2 through Java 26: Java 26 accepts class-file major versions **45–70**. ([Oracle Docs](#))

The Most Important Compatibility Rule

The easiest way to understand Java compatibility is this:

Direction	Example	Usually Works?	Why
Old .class → New JVM	Java 8 .class → Java 26 JVM	✓ Yes	New JVMs retain support for older class-file versions.

Old source → New compiler	Java 8 source → Java 26 compiler	✓ Usually	Java maintains strong source compatibility, although individual incompatibilities exist.
New .class → Old JVM	Java 26 .class → Java 8 JVM	✗ No	Java 8 does not understand class-file version 70.
New source → Old compiler	Java 21 source → Java 8 compiler	✗ No	Java 8 compiler does not understand newer syntax/APIs.
Old API → New JDK	Java 8 ArrayList → Java 26	✓ Usually	Java maintains extensive API compatibility.
JDK-internal API → New JDK	sun.misc.* / internal APIs	☐ May fail	Internal APIs are not guaranteed stable.
Preview feature → Later JDK	Java 21 preview feature → Java 26	☐ Not automatically	Preview features are release-specific unless standardized.
New feature → Old JDK	Java 26 primitive pattern → Java 8	✗	Old JDK has no compiler/JVM support for it.

Oracle's JLS explicitly defines binary-compatible changes such as adding methods/fields and reimplementing methods, while noting that platform-release compatibility is a separate issue. ([Oracle Docs](#))

Class-File Compatibility: Java 26 → Java 1

This is particularly important when discussing **backward compatibility**.

Java	Class-file Major Version	Can Java 26 JVM read it?
Java 1.0.2	45	✓
Java 1.1	45	✓

Java 1.2	46	✓
Java 1.3	47	✓
Java 1.4	48	✓
Java 5	49	✓
Java 6	50	✓
Java 7	51	✓
Java 8	52	✓
Java 9	53	✓
Java 10	54	✓
Java 11	55	✓
Java 12	56	✓
Java 13	57	✓
Java 14	58	✓
Java 15	59	✓
Java 16	60	✓
Java 17	61	✓
Java 18	62	✓
Java 19	63	✓
Java 20	64	✓
Java 21	65	✓
Java 22	66	✓
Java 23	67	✓
Java 24	68	✓
Java 25	69	✓
Java 26	70	✓

This table comes directly from the Java SE 26 JVM specification. ([Oracle Docs](#))

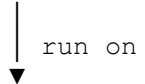
For example:

```

Java 8
  |
  | compile
  ▼
Hello.class

```

(class-file version 52)



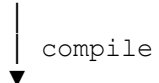
Java 26 JVM



Works in principle

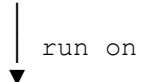
But the reverse is different:

Java 26



Hello.class

(class-file version 70)



Java 8 JVM



✗ Unsupported class-file version

Feature-Level Compatibility

A feature can be backward compatible in one sense but incompatible in another.

Example 1 — Lambda

Java 8:

```
List<String> names = List.of("A", "B", "C");
```

```
names.forEach(name -> System.out.println(name));
```

The **Java 7 compiler cannot compile this source**, because lambda expressions did not exist in Java 7.

Therefore:

Property	Result
Java 8 source → Java 7 compiler	✗
Java 8 .class → Java 7 JVM	✗
Java 7 .class → Java 8 JVM	✓ generally
Java 7 source → Java 8 compiler	✓ generally

Java 8 introduced lambda expressions, streams, functional interfaces and related APIs. ([Oracle Docs](#))

Example 2 — Java 7 try-with-resources

```
try (FileInputStream input =  
    new FileInputStream("data.txt")) {  
  
    // use input  
}
```

This is valid Java 7+, but a Java 6 compiler does not understand the syntax.

Therefore:

New syntax is not forward-compatible with older compilers.

Example 3 — Java 10 var

```
var name = "Anurag";  
var age = 43;
```

The compiler infers:

```
name → String  
age  → int
```

But Java 8 cannot compile this syntax.

So:

Java 10 feature → Java 8 compiler = ✗

Example 4 — Java 16 Record

```
record Employee(int id, String name) {}
```

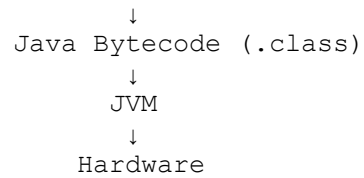
Java 16 introduced records as a permanent language feature. Java 8/11 cannot compile this source.

But a Java 26 JVM can generally execute older Java class files because Java 26 supports class-file versions going back to 45. ([Oracle Docs](#))

Why Java Has Such Strong Backward Compatibility

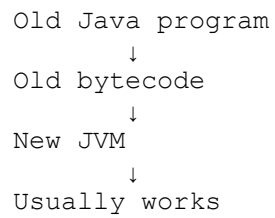
Java deliberately separates:

```
Java Source Code  
    ↓  
javac
```



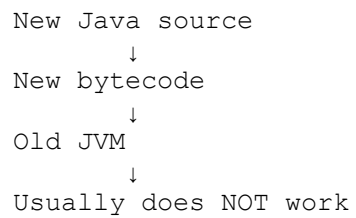
The JVM is designed to preserve compatibility with older bytecode.

Consequently:



This is one of Java's major strengths.

However:



Major Compatibility Breakpoints

Release	Compatibility Importance	Reason
---------	--------------------------	--------

Java 1.1	Medium	Major API additions and inner classes
Java 1.2	Medium	Collections, Swing and Java 2 platform
Java 1.4	Medium	Assertions, NIO, regex, logging
Java 5	High	Generics, annotations, enums, autoboxing, enhanced for
Java 6	Low for language	No Java-language changes; substantial API/runtime improvements. (Oracle Docs)
Java 7	Medium	try-with-resources, diamond, String switch, multi-catch
Java 8	Very High	Lambdas, streams, default methods, functional programming
Java 9	Very High	JPMS/modules and stronger encapsulation
Java 11	Medium/High	Removed/deprecated legacy components and newer APIs
Java 17	High	Strong encapsulation and sealed classes
Java 21	High feature impact	Virtual threads, record patterns, pattern matching
Java 25	Medium	New permanent language conveniences
Java 26	Medium currently	Primitive patterns are still preview in Java 26. (Oracle Docs)

Bottom Line

Java's compatibility model can be summarized as:

Question	Answer
Does newer Java generally run older Java bytecode?	Yes
Does older Java run newer Java bytecode?	No
Does newer Java generally compile older Java source?	Yes
Does older Java compile newer Java source?	No
Are all old APIs guaranteed forever?	No
Are JDK-internal APIs guaranteed?	No
Are preview features permanently compatible?	No
Can behavior change despite source/binary compatibility?	Yes
Is Java 26 backward-compatible with Java 1.x bytecode at the JVM level?	Yes, subject to the limits of the supported class-file format and runtime behavior
Is Java 1.x forward-compatible with Java 26 features?	No

The key distinction is therefore **backward compatibility** $\neq$ “**every old application will always work without modification.**” Oracle’s migration documentation explicitly warns that every new Java SE release can introduce binary, source, and behavioral incompatibilities, with JDK 9 being a particularly significant transition because of modularization and encapsulation. ([Oracle Docs](#))

If by “**all Java features**” you mean *every individual language feature, JVM feature, API, library, tool, JEP and feature-removal from Java 26 all the way back to Java 1.0*, that would be a much larger catalog than the major-feature table above; the table here focuses on the **major Java SE language/platform features and their compatibility characteristics**.

Java Class File Major Version

A class file's major version is a number stored inside every compiled Java .class file that tells the JVM which version of the Java class-file format was used to create that file.

In simple terms:

Class-file major version = the JVM bytecode format version of a .class file.

The JVM checks this version before loading a class. If the class was compiled for a newer Java version than the JVM supports, you commonly get:

UnsupportedClassVersionError

1. Basic Concept

Term	Definition	Example
Java source file	Human-readable Java program	Hello.java
Java compiler (javac)	Converts Java source into bytecode	javac Hello.java
Class file	Binary file containing JVM bytecode	Hello.class
Class-file format	Specification describing how .class files are structured	JVM specification
Major version	Identifies the class-file format version	Java 8 → 52
Minor version	Additional version component used primarily for special/preview formats	Java 8 → 0
JVM	Loads and executes class files	Java 26 JVM

UnsupportedClassVersionError	Error when JVM cannot support the class-file version	Java 8 JVM loading Java 26 class
-------------------------------------	--	----------------------------------

2. Where Is the Major Version Stored?

A Java class file begins with a fixed header:

```

+-----+
| Magic Number          | 4 bytes
+-----+
| Minor Version         | 2 bytes
+-----+
| Major Version         | 2 bytes
+-----+
| Constant Pool Count   |
+-----+
| Constant Pool         |
+-----+
| Access Flags          |
+-----+
| ...                   |
+-----+
```

The first four bytes are:

CA FE BA BE

These are called the **magic number**.

For example, a Java 8 class file begins conceptually like:

```

CA FE BA BE
00 00
00 34
```

...

Here:

Bytes	Meaning	Decimal
CA FE BA BE	Magic number	—
00 00	Minor version	0
00 34	Major version	52

Because hexadecimal 34 equals decimal 52, the class-file major version is **52**.

3. Major Version Mapping

The commonly used Java class-file major versions are:

Java Version	Class-File Major Version	Hexadecimal	Example
Java 1.0.2	45	0x2D	Very old class file
Java 1.1	45	0x2D	Hello.class
Java 1.2	46	0x2E	Java 2
Java 1.3	47	0x2F	Java 1.3
Java 1.4	48	0x30	Java 1.4
Java 5	49	0x31	Generics-era Java

Java 6	50	0x32	Java 6
Java 7	51	0x33	Java 7
Java 8	52	0x34	Lambdas/Streams
Java 9	53	0x35	Modules
Java 10	54	0x36	var
Java 11	55	0x37	Java 11
Java 12	56	0x38	Java 12
Java 13	57	0x39	Java 13
Java 14	58	0x3A	Java 14
Java 15	59	0x3B	Java 15
Java 16	60	0x3C	Records
Java 17	61	0x3D	Sealed classes
Java 18	62	0x3E	Java 18
Java 19	63	0x3F	Virtual threads preview
Java 20	64	0x40	Java 20
Java 21	65	0x41	Virtual threads
Java 22	66	0x42	Java 22
Java 23	67	0x43	Java 23

Java 24	68	0x44	Java 24
Java 25	69	0x45	Java 25
Java 26	70	0x46	Java 26

Java SE 26 specifies class-file major versions through **70** and supports older class-file versions in the range defined by the JVM specification.

4. Why Does the JVM Need It?

The JVM needs to know **which class-file format it is reading**.

For example:

```

Java 26 compiler
  ↓
Hello.java
  ↓
Hello.class
  ↓
Major Version = 70
  ↓
Java 26 JVM
  ↓
✓ Understands it

```

But:

```

Java 26 compiler
  ↓
Hello.class
  ↓

```

Major Version = 70

↓
Java 8 JVM

↓
✗ Does not understand version 70

The JVM therefore rejects the class.

5. UnsupportedClassVersionError

Suppose you compile:

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello Java");  
    }  
}
```

using Java 26:

```
javac Hello.java
```

The resulting class file has approximately:

Major version = 70

Now try to execute it with Java 8.

Conceptually:

```
Java 26  
|
```

```
| Hello.class
| major = 70
▼
Java 8 JVM
|
| supports approximately up to major = 52
▼
ERROR
```

You can receive an error similar to:

```
java.lang.UnsupportedClassVersionError:
Hello has been compiled by a more recent version
of the Java Runtime
```

6. Java 8 Example

Suppose:

JDK = Java 8

and you compile:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello");
    }
}
```

The resulting class file has:

Major version = 52

Therefore:

Compiler	Class-file major	JVM	Result
Java 8	52	Java 8	✓
Java 8	52	Java 11	✓
Java 8	52	Java 17	✓
Java 8	52	Java 21	✓
Java 8	52	Java 26	✓ Generally

This illustrates **backward compatibility of newer JVMs with older class files**.

7. Reverse Direction

Now compile using Java 21:

```
Java 21
  ↓
Hello.class
  ↓
Major = 65
```

Try running it on Java 8:

```
Java 8 JVM
  ↓
Expected maximum class-file version ≈ 52
  ↓
Received version = 65
```

✗ ↓
UnsupportedClassVersionError

Therefore:

Direction	Result
Java 8 class → Java 21 JVM	✓
Java 8 class → Java 26 JVM	✓ Generally
Java 21 class → Java 8 JVM	✗
Java 26 class → Java 8 JVM	✗

8. How to Check a Class File's Major Version

The easiest standard JDK tool is:

```
javap -verbose Hello.class
```

You may see:

```
Classfile /.../Hello.class
  Last modified ...
  major version: 52
  minor version: 0
```

Therefore:

major version: 52

means:

Java 8

9. javap Example

Suppose you have:

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello");  
    }  
}
```

Compile:

```
javac Hello.java
```

Inspect:

```
javap -verbose Hello.class
```

Relevant output:

```
minor version: 0  
major version: 52
```

Interpretation:

Output	Meaning
minor version: 0	Minor class-file version
major version: 52	Java 8 class-file format

--	--

10. Major Version vs Java Version

An important point is that the numbers **do not directly match**.

For example:

Java	Major Version
Java 8	52
Java 11	55
Java 17	61
Java 21	65
Java 25	69
Java 26	70

So:

Java 8 ≠ major version 8

Instead:

Java 8 → major 52

and:

Java 26 → major 70

11. Major Version vs Minor Version

A class file contains **both**:

minor_version
major_version

For a typical class:

minor version: 0
major version: 65

means:

Java 21 class-file format

The major version identifies the principal class-file format, while the minor version can distinguish special variants. Modern Java also uses special minor-version values for some **preview** class files.

Field	Purpose
Major version	Main class-file format version
Minor version	Secondary/special version information
Magic number	Identifies the file as a Java class file

12. Relationship With --release

You can deliberately compile code for an older Java platform.

For example, using a modern JDK:

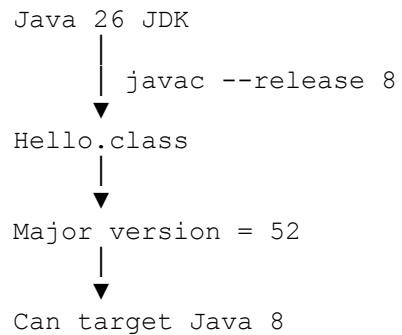
```
javac --release 8 Hello.java
```

The compiler produces a Java 8-compatible class file:

```
major version = 52
```

Even if the compiler itself is Java 26.

Conceptually:



This is extremely useful when deploying an application to an older Java runtime.

13. --release vs -source / -target

Option	Purpose
--release 8	Targets Java 8 language, API and class-file format
-source 8	Controls source-language level

-target 8	Controls generated class-file target
javac without these	Uses the compiler's default/current target

For modern builds, --release is generally preferable when you need to target an older Java release because it also restricts access to APIs introduced after that release.

Example:

```
javac --release 8 Hello.java
```

produces Java 8-compatible output.

14. Class File Is Not the Same as Java Source

Consider:

```
List<String> names = new ArrayList<>();
```

The source code contains Java-language syntax.

After compilation:

```
Hello.java
  ↓
javac
  ↓
Hello.class
  ↓
JVM bytecode
```

The .class file does **not** simply contain your original Java source.

It contains:

- JVM instructions
- constant pool
- class metadata
- method information
- field information
- attributes
- version information

The major version tells the JVM which class-file format rules apply.

15. Complete Class-File Header

Conceptually:

Magic Number CA FE BA BE
Minor Version 2 bytes
Major Version 2 bytes
Constant Pool Count
Constant Pool
Access Flags
This Class

Super Class
Interfaces
Fields
Methods
Attributes

The JVM specification defines this class-file structure.

16. Practical Compatibility Table

Class Major Version	Corresponding Java	Java 8 JVM	Java 17 JVM	Java 21 JVM	Java 26 JVM
52	Java 8	✓	✓	✓	✓
55	Java 11	✗	✓	✓	✓
61	Java 17	✗	✓	✓	✓
65	Java 21	✗	✗	✓	✓
69	Java 25	✗	✗	✗	✓
70	Java 26	✗	✗	✗	✓

This demonstrates the fundamental rule:

A JVM can normally run class files from its own release and older supported releases, but not class files compiled for a newer release.

17. Simple Real-World Example

Imagine a company has:

Production Server
Java 17

but a developer uses:

Development Machine
Java 21

The developer compiles:

```
javac Hello.java
```

The class file will have:

major version = 65

The Java 17 server expects a Java 17-compatible class file:

maximum supported major $\approx$ 61

Therefore:

```
Java 21 compiled class  
      |  
      v major 65
```

Java 17 JVM



✗ `UnsupportedClassVersionError`

The solution is to compile for Java 17:

```
javac --release 17 Hello.java
```

Now:

major version = 61

and:

Java 17 class



Java 17 JVM



18. Key Points to Remember

Question	Answer
What is a class-file major version?	A number identifying the JVM class-file format version.
Where is it stored?	In the class-file header.
How large is it?	2 bytes.
What comes before it?	Magic number and minor version.
What is Java 8's major version?	52

What is Java 17's major version?	61
What is Java 21's major version?	65
What is Java 25's major version?	69
What is Java 26's major version?	70
How can I inspect it?	<code>javap -verbose MyClass.class</code>
What happens when JVM sees an unsupported newer version?	<code>UnsupportedClassVersionError</code>
Can Java 26 generally run Java 8 class files?	Yes
Can Java 8 run Java 26 class files?	No
Can <code>javac --release 8</code> produce Java 8-compatible classes from a modern JDK?	Yes
Does major version equal Java version?	No
Is class-file version the same as source version?	No

One-line definition

A Java class file's major version is the numeric identifier in the .class file header that tells the JVM which major version of the JVM class-file format the bytecode conforms to—for example, Java 8 uses major version 52 and Java 26 uses major version 70.

Yes. The key idea is:

You normally do not need to recompile an old .class file to run it on a newer Java JVM. A newer JVM is generally designed to understand older class-file major versions.

For example, a Java 8 class file (major version **52**) can generally be run directly on Java 26 (major version **70**).

However, if you have **old Java source code**, you may want to recompile it with a modern JDK. That is a different process.

1. Three different situations

Situation	Example	What you should do
You have an old .class file	Java 8 .class → Java 26 JVM	Run it directly
You have old .java source	Java 8 source → Java 26 JDK	Recompile it
You want Java 26 compiler but Java 8-compatible output	Java 26 javac → Java 8 .class	Use --release 8
You have a Java 26 .class and want Java 8	Java 26 .class → Java 8 JVM	✗Cannot simply run it
Old class uses removed/encapsulated JDK APIs	Old application → Java 26	May require code changes/migration

2. Java versions and class-file major versions

Java	Year	Major Version	Can Java 26 JVM normally load it?
Java 1.0.2	1996	45	✓
Java 1.1	1997	45	✓
Java 1.2	1998	46	✓
Java 1.3	2000	47	✓
Java 1.4	2002	48	✓
Java 5	2004	49	✓
Java 6	2006	50	✓
Java 7	2011	51	✓
Java 8	2014	52	✓
Java 9	2017	53	✓
Java 10	2018	54	✓
Java 11	2018	55	✓
Java 12	2019	56	✓
Java 13	2019	57	✓
Java 14	2020	58	✓
Java 15	2020	59	✓
Java 16	2021	60	✓
Java 17	2021	61	✓
Java 18	2022	62	✓
Java 19	2022	63	✓
Java 20	2023	64	✓
Java 21	2023	65	✓
Java 22	2024	66	✓
Java 23	2024	67	✓
Java 24	2025	68	✓
Java 25	2025	69	✓
Java 26	2026	70	✓

Java SE 26's JVM specification defines the class-file format and supported version range.

3. The easiest case: old .class → Java 26

Suppose you have:

Hello.class

and javap reports:

major version: 52

That means:

major 52 → Java 8

You can install/use Java 26 and try:

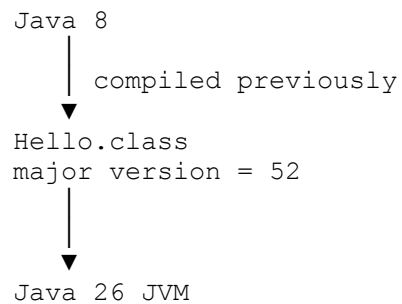
java Hello

You **do not** do:

javac Hello.class

because javac compiles **source code**, not an ordinary .class file.

The flow is:



```
major version supported = 70
```



RUN

4. Complete example

Step 1 — Create old source

Suppose the original source was:

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello from old Java!");  
    }  
}
```

Step 2 — Compile using Java 8

On a Java 8 JDK:

```
javac Hello.java
```

You get:

```
Hello.class
```

Its class-file version is:

```
52
```

Step 3 — Move only the .class file

For example:

```
old/  
Hello.class
```

Step 4 — Use Java 26

Check:

```
java -version
```

You should see a Java 26 runtime.

Step 5 — Run

```
java Hello
```

Expected:

Hello from old Java!

No recompilation is necessary.

5. How to inspect an old class file

Use:

```
javap -verbose Hello.class
```

Look for:

minor version: 0

major version: 52

Therefore:

52 → Java 8

You can use the following conversion:

Major	Java
45	1.0/1.1
46	1.2
47	1.3
48	1.4
49	5
50	6
51	7
52	8
53	9
54	10
55	11
56	12
57	13
58	14
59	15
60	16
61	17
62	18
63	19
64	20
65	21
66	22
67	23
68	24

69	25
70	26

6. What if you have the old .java source?

This is different.

Suppose you have:

Hello.java

written in Java 8.

You can compile it with Java 26:

```
javac Hello.java
```

The resulting .class file will normally target the current compiler's default release, rather than preserving the original Java 8 class-file version.

If you want the output to remain Java 8-compatible, use:

```
javac --release 8 Hello.java
```

Then:

```
Java 26 javac
      |
      | --release 8
      v
Hello.class
      |
      v
major version = 52
```

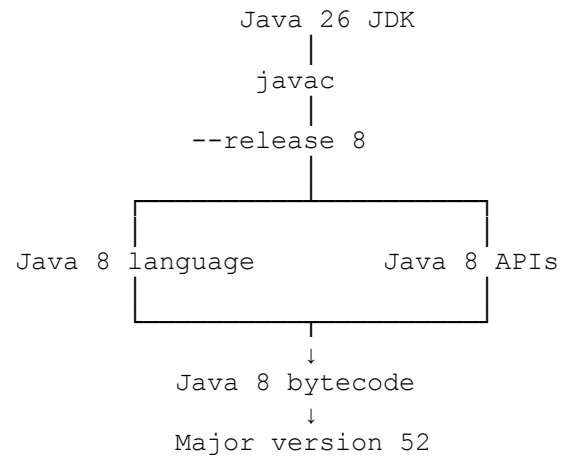
7. Why --release is important

Consider:

```
javac --release 8 Hello.java
```

The compiler is Java 26, but you're asking it to produce a Java 8-targeted program.

Conceptually:



This is preferable to merely changing the class-file target because `--release` also restricts the available Java API surface to the selected release.

8. Old source → Java 26

Suppose you have a Java 5 application:

```
public class Employee {  
    public static void main(String[] args) {  
        System.out.println("Employee application");  
    }  
}
```

You can attempt:

```
javac Employee.java
```

with Java 26.

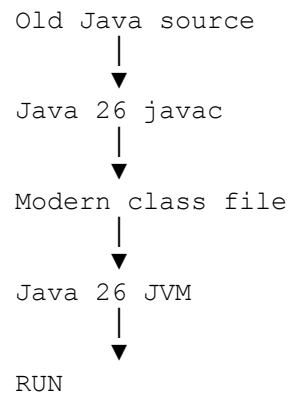
If the source uses language constructs that remain supported, it may compile successfully.

Then:

```
java Employee
```

runs on Java 26.

The flow is:



This is called **source migration/recompilation**, not merely running an old class file.

9. Old .class → Java 26 does NOT mean every application will work

This is an extremely important distinction.

Suppose:

```
OldApplication.class  
major version = 49
```

Java 26 can understand the old class-file format.

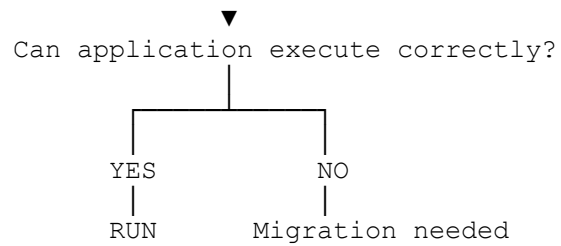
But the application might depend on:

```
old external libraries  
old JDK APIs  
removed components  
JDK internal APIs  
obsolete configuration  
operating-system assumptions  
old security behavior
```

Therefore:

Can JVM understand class file?

↓
▼
YES
|



Class-file compatibility does not guarantee application compatibility.

10. Example: Java 8 application on Java 26

Imagine:

```
Application
├── MyApp.class      major 52
├── Database.class   major 52
├── Utils.class      major 52
└── library.jar
```

You copy it to a Java 26 machine.

Run:

```
java -cp ./library.jar MyApp
```

The JVM can generally load the Java 8 class files.

But you might encounter:

```
ClassNotFoundException
NoClassDefFoundError
```

IllegalAccessError
InaccessibleObjectException
UnsupportedClassVersionError

These errors have **different causes**.

Error	Possible issue
UnsupportedClassVersionError	Class is newer than JVM
ClassNotFoundException	Required class not found
NoClassDefFoundError	Required class/library unavailable
IllegalAccessError	Access/linkage incompatibility
InaccessibleObjectException	Strong encapsulation/access restrictions
NoSuchMethodError	Binary/API incompatibility

11. Running a Java 1.4 class on Java 26

Suppose:

OldApp.class
major version = 48

This corresponds to Java 1.4.

You can try:

`java OldApp`

with Java 26.

Conceptually:

```
Java 1.4
  ↓
OldApp.class
  ↓
major 48
  ↓
Java 26 JVM
  ↓
Understands old class-file format
  ↓
Attempts execution
```

If the program only uses functionality still available and compatible with Java 26, it may run.

If it relies on removed/changed functionality, it may fail.

12. Running a Java 7 class on Java 26

Java 7:

major = 51

Java 26:

major = 70

Therefore:

$51 < 70$

The Java 26 JVM understands the older class-file format.

Example:

```
java MyJava7Application
```

No conversion of MyJava7Application.class is required merely because you're using Java 26.

13. Running a Java 26 class on Java 8

Now reverse the situation.

Java 26:

```
major = 70
```

Java 8:

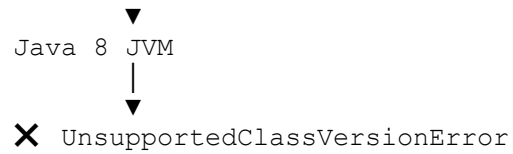
```
major = 52
```

Therefore:

$70 > 52$

Java 8 cannot understand the Java 26 class-file format.

```
Java 26 .class
major = 70
|
```



You cannot fix this simply by changing:

70 → 52

with a hex editor.

Why?

Because newer bytecode may contain:

- newer JVM instructions
- newer class-file structures
- newer constant-pool forms
- newer attributes
- newer language semantics

Simply changing the number would not magically transform the bytecode into Java 8 bytecode.

14. Correct way to produce Java 8-compatible output

If you have source:

Hello.java

and Java 26:

```
javac --release 8 Hello.java
```

Then:

```
Hello.class  
major version = 52
```

Now it can target a Java 8 runtime, subject to the program using APIs and behavior available in Java 8.

15. Java 1 → Java 26 migration strategy

If you have an extremely old Java application, don't immediately assume that installing Java 26 is enough.

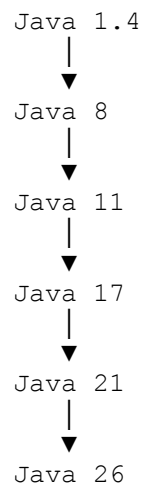
Use a staged process.

Stage	Action	Purpose
1	Identify application Java version	Determine starting point
2	Inspect .class files	Determine major versions
3	Identify dependencies	Find old JARs/libraries
4	Run on current Java	Discover runtime problems
5	Compile from source if available	Modernize source
6	Fix removed APIs	Replace obsolete APIs
7	Fix encapsulation problems	Replace internal JDK APIs
8	Update third-party libraries	Obtain Java 26-compatible versions

9	Test application	Verify behavior
10	Deploy	Move to Java 26

16. Recommended migration path for very old applications

For a Java 1.4 application, for example:



However, **you don't necessarily have to compile separately at every intermediate version**. The appropriate path depends on the application, dependencies, build system, and APIs it uses.

For a sufficiently old application, intermediate versions can be useful as **diagnostic migration stages**.

17. Maven example

Suppose the source code is old, but you want to compile it for Java 17 using a modern JDK.

In Maven, you can configure:

```
<properties>
  <maven.compiler.release>17</maven.compiler.release>
</properties>
```

Then:

```
mvn clean package
```

The compiler produces Java 17-targeted class files:

```
major version = 61
```

Similarly:

```
<maven.compiler.release>8</maven.compiler.release>
```

targets:

```
major version = 52
```

18. Gradle example

With Gradle, a modern configuration can use:

```
java {
  toolchain {
```

```
    languageVersion = JavaLanguageVersion.of(17)
  }
}
```

Or configure compilation compatibility appropriately for the required target.

The important principle is:

```
Source
  ↓
Compiler
  ↓
Target Java release
  ↓
Class-file major version
```

19. Multiple classes must be considered

Suppose you have:

```
Application.class → 52
Database.class   → 52
Utility.class    → 61
```

This is a mixed application.

Running on Java 17:

```
52 → Java 8 class → ✓
61 → Java 17 class → ✓
```

Running on Java 8:

```
52 → ✓
```

61 → ✕

Therefore, **the newest class-file version used by the application matters.**

20. JAR files

A .jar file is essentially a ZIP-format archive that can contain:

```
application.jar
├── META-INF/
├── com/
│   └── example/
│       ├── Main.class
│       ├── User.class
│       └── Database.class
└── application.properties
```

The JAR itself doesn't have one universal Java major version.

Individual .class files inside it have class-file versions.

Therefore:

```
JAR
├── A.class → 52
├── B.class → 52
├── C.class → 61
└── D.class → 65
```

The Java runtime must be capable of loading whichever classes are actually used.

21. Practical command sequence

Suppose you receive:

legacy-application.jar

Step 1 — Check Java 26

```
java -version
```

Step 2 — Inspect classes

```
jar tf legacy-application.jar
```

Step 3 — Extract if necessary

```
jar xf legacy-application.jar
```

Step 4 — Inspect a class

```
javap -verbose com/example/Main.class
```

Look for:

major version: 52

Step 5 — Run

```
java -jar legacy-application.jar
```

If the application starts, you have successfully run the old class files on Java 26.

22. What NOT to do

Incorrect approach	Why
javac OldProgram.class	javac expects Java source, not ordinary .class files
Change major version 70 to 52 manually	Does not convert bytecode
Assume every Java 1.x application works on Java 26	APIs/dependencies/behavior may have changed
Assume old JAR = old Java version	A JAR can contain classes compiled for different releases
Recompile without testing dependencies	Third-party libraries may be incompatible
Replace every old class-file version manually	Class-file compatibility is more than a single number

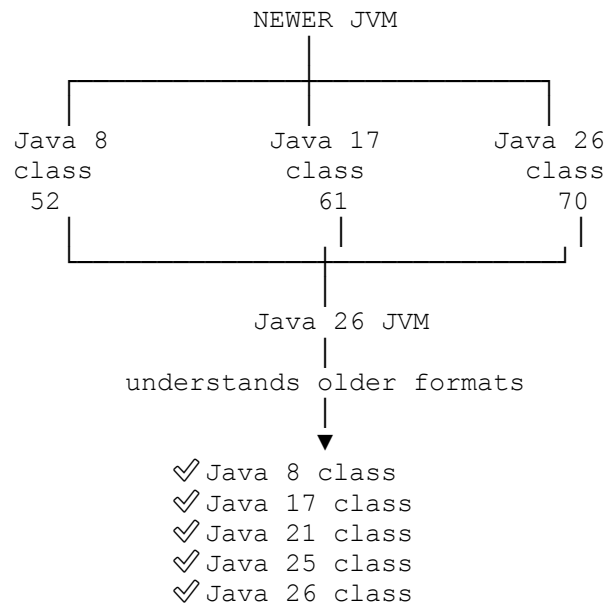
23. The most important distinction

You have	Goal	Correct approach
Java 8 .class	Run on Java 26	Run directly
Java 7 .class	Run on Java 26	Run directly
Java 1.4 .class	Run on Java 26	Try directly; migrate if application issues occur
Java 8 .java	Run on Java 26	Compile with Java 26
Java 8 .java	Keep Java 8 compatibility	javac --release 8
Java 26 .java	Run on Java 8	Rewrite/use compatible source and compile with --release 8; newer APIs/features cannot

		simply be used
Java 26 .class	Run on Java 8	✗ Recompile from source for Java 8
Old .class, no source	Modernize for Java 26	Try direct execution first; if incompatible, source recovery/replacement may be necessary

24. Final mental model

Think of Java class-file versions like **language levels understood by a JVM**:



But in the opposite direction:

Java 26 class
major = 70



Java 8 JVM
major support $\approx$ 52



✗ `UnsupportedClassVersionError`

The rule to remember

To run an old .class file on Java 26, normally just run it with the Java 26 JVM. You do not need to recompile it merely because its major version is old. If you have the source and want to produce a class file compatible with an older Java version, use the modern JDK's `javac --release N` option.

And one final distinction: **“Java 26 can load an old class file” does not guarantee that the old application itself will work unchanged.** Class-file compatibility, API compatibility, binary compatibility, and behavioral compatibility are separate issues.

Yes—but there is an important limitation:

You cannot take a Java 26 .class file (major version 70) and simply “convert” it into a Java 1.x/old-Java .class file.

To run on an older JVM, you normally need the **Java source code** and must compile it for the desired older Java release using `--release`, provided the source and APIs are compatible with that release.

The direction is therefore:

```
Java 26 source
  ↓
Java 26 javac
  ↓
--release N
  ↓
Older-compatible .class
```

↓
Older JVM

not:

Java 26 .class
↓
change 70 → 52
↓
Java 8

Changing the number alone does **not** convert the bytecode.

1. Java 26 → Older Java: The Core Concept

Term	Definition	Example
Java 26 source	Java source written using Java 26 language/API capabilities	MyApp.java
Java 26 class file	Bytecode produced for Java 26	Major version 70
Target Java version	The oldest Java release you want the application to run on	Java 8
--release	javac option that targets a specific Java release	javac --release 8 MyApp.java
Target class file	Class file produced for the selected older release	Java 8 → major 52
Older JVM	JVM that executes the generated class	Java 8 JVM
Backward compatibility	Newer Java can generally work with	Java 26 JVM → Java 8 class

	older class files	
Forward compatibility	Older Java can directly execute newer Java bytecode	✗ Generally not supported

2. Java 26 Class-File Version

Java 26 uses:

```
Major version = 70
```

Therefore:

```
Java 26
  ↓
javac
  ↓
MyApp.class
  ↓
major version = 70
```

An older JVM such as Java 8 understands class files only up to its supported class-file format:

```
Java 8 → major version 52
```

Therefore:

```
Java 26 class
major = 70
  ↓
```

Java 8 JVM



✗ `UnsupportedClassVersionError`

3. Complete Java 26 → Older-Version Mapping

Target Java	Target Class-File Major	Java 26 <code>javac --release</code>	Can Java 26 source use all Java 26 features?	Can resulting class run on target JVM?*
Java 25	69	--release 25	✗ No	✓ If source/API compatible
Java 24	68	--release 24	✗ No	✓
Java 23	67	--release 23	✗ No	✓
Java 22	66	--release 22	✗ No	✓
Java 21	65	--release 21	✗ No	✓
Java 20	64	--release 20	✗ No	✓
Java 19	63	--release 19	✗ No	✓
Java 18	62	--release 18	✗ No	✓
Java 17	61	--release 17	✗ No	✓
Java 16	60	--release 16	✗ No	✓
Java 15	59	--release 15	✗ No	✓
Java 14	58	--release 14	✗ No	✓
Java 13	57	--release 13	✗ No	✓
Java 12	56	--release 12	✗ No	✓
Java 11	55	--release 11	✗ No	✓
Java 10	54	--release 10	✗ No	✓
Java 9	53	--release 9	✗ No	✓
Java 8	52	--release 8	✗ No	✓
Java 7	51	--release 7*	✗ No	□ Depends on JDK/compiler support

Java 6	50	--release 6*	✗ No	☐ Modern JDK limitations
Java 5	49	--release 5*	✗ No	☐ Modern JDK limitations
Java 1.4	48	Not supported by modern --release	✗	☐ Requires an older toolchain
Java 1.3	47	Not supported by modern --release	✗	☐ Requires an older toolchain
Java 1.2	46	Not supported by modern --release	✗	☐ Requires an older toolchain
Java 1.1	45	Not supported by modern --release	✗	☐ Requires an older toolchain
Java 1.0/1.0.2	45	Not supported by modern --release	✗	☐ Requires historical toolchain

*The exact lowest releases supported by a particular modern JDK's --release option depend on the JDK. Modern JDKs generally provide supported release targets from Java 6/7/8 onward depending on the compiler version; **Java 26 should not be assumed to produce Java 1.0–5 class files directly.**

4. The Most Important Command

For Java 26 → Java 8:

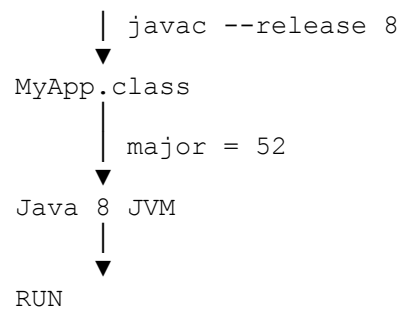
```
javac --release 8 MyApp.java
```

This means:

"Use this modern compiler, but produce output compatible with Java 8."

The result is approximately:

```
Java 26 JDK
|
```



5. Example: Java 26 Source That Can Be Backported

Suppose your source is:

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello Java");  
    }  
}
```

This program doesn't require modern Java language features.

Compile with Java 26:

```
javac --release 8 Hello.java
```

Check:

```
javap -verbose Hello.class
```

You should find:

major version: 52

Now:

Java 26 compiler



--release 8



major 52



Java 8 JVM



6. Example: Java 26 Feature That Cannot Be Backported Automatically

Suppose you write:

```
public class Example {  
    public static void main(String[] args) {  
        var name = "Anurag";  
        System.out.println(name);  
    }  
}
```

var was introduced in Java 10.

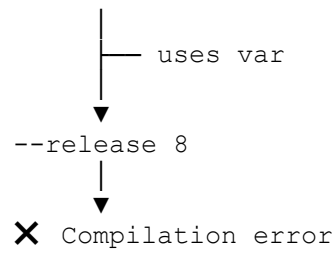
Now execute:

```
javac --release 8 Example.java
```

The compiler will reject it because Java 8 doesn't support var.

Conceptually:

Java 26 source



You must rewrite it:

```
public class Example {  
    public static void main(String[] args) {  
        String name = "Anurag";  
        System.out.println(name);  
    }  
}
```

Then:

```
javac --release 8 Example.java
```

works.

7. Java 26 Lambda vs Java 7

Consider:

```
List<String> names =  
    List.of("A", "B", "C");
```

```
names.forEach(System.out::println);
```

There are multiple problems if targeting Java 7:

Code	Introduced
Lambda expressions	Java 8
List.of()	Java 9
Method references	Java 8
forEach() default method	Java 8

Therefore this cannot simply become Java 7 bytecode.

You must rewrite it using older APIs:

```
List<String> names =  
    Arrays.asList("A", "B", "C");  
  
for (String name : names) {  
    System.out.println(name);  
}
```

Then use an appropriate older JDK/toolchain to target Java 7.

8. Java 26 → Java 17

This is a very common modern migration.

Suppose your source uses only Java 17-compatible features:

```
public class Employee {  
    private final int id;  
  
    public Employee(int id) {  
        this.id = id;  
    }  
  
    public int getId() {  
        return id;  
    }  
}
```

Compile:

```
javac --release 17 Employee.java
```

Result:

```
major version = 61
```

Then:

```
java Employee
```

on a Java 17 JVM can execute it.

9. Java 26 → Java 21

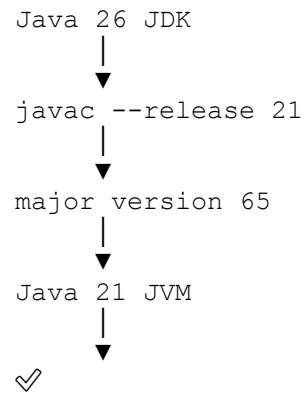
Same principle:

```
javac --release 21 MyApp.java
```

Result:

major version = 65

Flow:



But your source cannot rely on Java 22–26-only language/API features.

10. Why --release Is Better Than -target

Older approaches often used:

```
javac -source 8 -target 8 MyApp.java
```

Modern Java development generally prefers:

```
javac --release 8 MyApp.java
```

Option	Controls source language	Controls bytecode	Restricts APIs
-source 8	✓	✗	✗

-target 8	✗	✓	✗
-source 8 -target 8	✓	✓	✗
--release 8	✓	✓	✓

For example, this is dangerous:

```
javac -source 8 -target 8 MyApp.java
```

because you might accidentally use a Java 26 API that doesn't exist in Java 8.

--release 8 helps prevent that.

11. Java 26 .class Cannot Simply Be Downgraded

Suppose you already have:

```
MyApp.class
major version = 70
```

and you want:

```
major version = 52
```

You might think:

```
70 → 52
```

is enough.

It is not.

A class file contains much more than its version number.

```
Java 26 .class
├─ Magic number
├─ Major version = 70
├─ Constant pool
├─ JVM instructions
├─ Methods
├─ Fields
├─ Attributes
├─ Bootstrap methods
└─ Other metadata
```

A newer class may contain structures that an old JVM doesn't understand.

Therefore:

Change 70 → 52

↓

✗ Not a real downgrade

12. If You Have Only the Java 26 .class File

This is an important scenario.

What you possess	Can you normally target an old Java version?
Java 26 .java source	✓
Java 26 .class only	✗ Not normally
Java 26 .jar containing classes only	✗ Not normally
Java 26 source + dependencies	✓, subject to source/API compatibility

Java 26 .class + source reconstructed/decompiled	☐ Possible, but requires rewriting/recompilation

If you have only:

MyApp.class

you cannot normally use:

```
javac --release 8 MyApp.class
```

because javac does not convert arbitrary class files from version 70 to version 52.

You need source code or a suitable transformation/migration process.

13. Java 26 → Java 8

This is one of the most useful examples.

Desired result

Java 26 source

↓

Java 8-compatible class

↓

Java 8 JVM

Command

```
javac --release 8 MyApp.java
```

Verify

```
javap -verbose MyApp.class
```

Expected:

major version: 52

Run

On Java 8:

```
java MyApp
```

Result:

✓ Runs if the source/API/dependencies are Java 8-compatible.

14. Java 26 → Java 11

```
javac --release 11 MyApp.java
```

Result:

major version = 55

Run with:

```
java MyApp
```

on Java 11.

15. Java 26 → Java 17

```
javac --release 17 MyApp.java
```

Result:

major version = 61

16. Java 26 → Java 21

```
javac --release 21 MyApp.java
```

Result:

major version = 65

17. Java 26 → Java 25

```
javac --release 25 MyApp.java
```

Result:

major version = 69

18. Java 26 → Java 1.4

This is fundamentally different.

You cannot expect:

```
javac --release 4 MyApp.java
```

to work with a modern JDK.

Modern JDKs don't generally provide arbitrary historical Java 1.x targets through `--release`.

Instead, you would need a **historical Java 1.4 development environment/toolchain** and source compatible with Java 1.4.

Conceptually:

```
Modern source
  ↓
Rewrite source to Java 1.4
  ↓
Java 1.4-compatible compiler/toolchain
  ↓
major version 48
  ↓
Java 1.4 JVM
```

19. Java 26 → Java 1.0/1.1

This is even more restrictive.

Java 26 cannot simply compile modern Java source into a Java 1.0 class file.

You would need:

```
Java 26 application
  ↓
rewrite source
  ↓
remove modern language features
  ↓
remove modern APIs
```

↓
use historical Java libraries
↓
historical Java compiler
↓
historical class-file format

This is essentially **porting**, not ordinary compilation.

20. What Happens With Each Java Feature?

Suppose your Java 26 source contains:

Feature	Introduced	Can target Java 8?	What must happen?
Classes	Java 1.0	✓	No problem
Inheritance	Java 1.0	✓	No problem
Exceptions	Java 1.0	✓	No problem
Threads	Java 1.0	✓	No problem
Inner classes	Java 1.1	✓	No problem
Assertions	Java 1.4	✓	No problem
Generics	Java 5	✓	No problem
Enhanced for	Java 5	✓	No problem
Annotations	Java 5	✓	No problem

try-with-resources	Java 7	✗	Rewrite
Lambda	Java 8	✓	Supported
Stream API	Java 8	✓	Supported
var	Java 10	✗	Rewrite
Switch expressions	Java 14	✗	Rewrite
Records	Java 16	✗	Replace with classes
Sealed classes	Java 17	✗	Rewrite using older constructs
Pattern matching	Later Java	✗	Rewrite
Virtual threads	Java 21	✗	Use traditional threads/executors
New Java 26 features	Java 26	✗	Rewrite/remove

This demonstrates an important principle:

--release does not translate new language features into old language features. It only allows compilation when your source is already compatible with the target release.

21. Example: Record → Java 8

Java 26 source:

```
public record Person(String name, int age) {
}
```

Target:

Java 8

This won't work:

```
javac --release 8 Person.java
```

because records don't exist in Java 8.

Rewrite:

```
public final class Person {  
    private final String name;  
    private final int age;  
  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    public String name() {  
        return name;  
    }  
  
    public int age() {  
        return age;  
    }  
}
```

Then:

```
javac --release 8 Person.java
```

Now you can produce:

major version = 52

22. Example: var → Java 8

Java 26:

```
var name = "Anurag";  
var age = 43;
```

Java 8-compatible:

```
String name = "Anurag";  
int age = 43;
```

Then:

```
javac --release 8 Example.java
```

23. Example: Switch Expression → Java 8

Modern:

```
int result = switch (day) {  
    case 1 -> 100;  
    case 2 -> 200;  
    default -> 0;  
};
```

Java 8-compatible rewrite:

```
int result;
```

```
switch (day) {  
    case 1:
```

```
    result = 100;
    break;

case 2:
    result = 200;
    break;

default:
    result = 0;
}
```

Then compile:

```
javac --release 8 Example.java
```

24. Example: Virtual Thread → Java 8

Modern Java:

```
Thread.startVirtualThread(() -> {
    System.out.println("Hello");
});
```

Java 8 doesn't have virtual threads.

You would need something like:

```
new Thread(() -> {
    System.out.println("Hello");
}).start();
```

Then compile for Java 8:

```
javac --release 8 Example.java
```

The behavior and scalability characteristics are **not equivalent**, however.

25. Java 26 → Every Older Version

The practical process is:

Target	Strategy
Java 25	javac --release 25
Java 24	javac --release 24
Java 23	javac --release 23
Java 22	javac --release 22
Java 21	javac --release 21
Java 20	javac --release 20
Java 19	javac --release 19
Java 18	javac --release 18
Java 17	javac --release 17
Java 16	javac --release 16
Java 15	javac --release 15
Java 14	javac --release 14
Java 13	javac --release 13
Java 12	javac --release 12
Java 11	javac --release 11
Java 10	javac --release 10
Java 9	javac --release 9
Java 8	javac --release 8

Java 7 and older	Use an appropriate supported/historical compiler/toolchain rather than assuming Java 26 --release can target it
Java 1.4 and older	Historical compiler/toolchain + source porting

26. The "Oldest Possible Java" Question

There are actually **three different meanings** of "oldest Java."

Meaning	Answer
Oldest class-file format Java 26 JVM can generally read	Very old class-file formats, including major version 45
Oldest Java release Java 26 javac can compile for using --release	Limited by the JDK's supported --release targets, not Java 1.0
Oldest Java release your modern Java 26 application can realistically be ported to	Depends entirely on its source, APIs, dependencies and required functionality

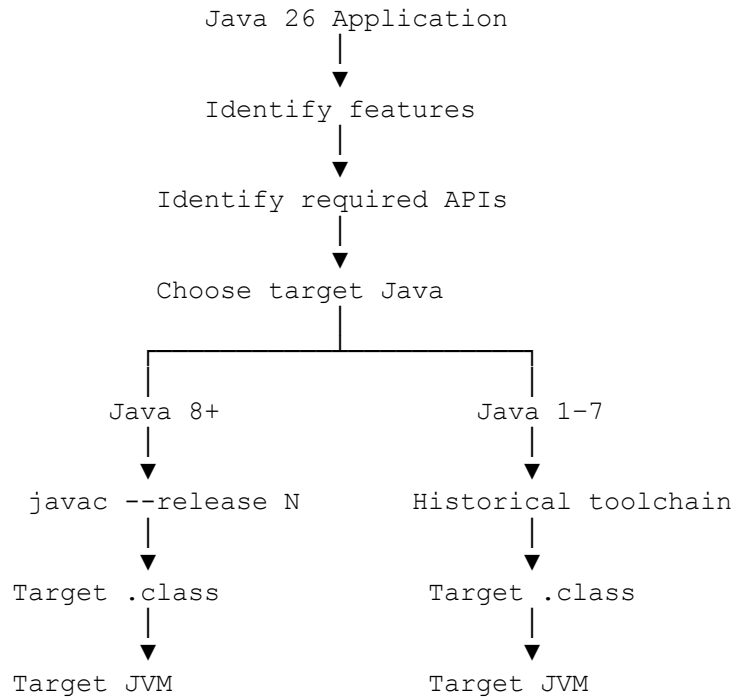
This distinction is critical.

27. Best Strategy

If your objective is:

"I have a Java 26 application and want it to run on the oldest practical Java version."

Use this approach:



28. Recommended Target Versions

For practical modern applications:

Goal	Recommended target
Maximum compatibility with reasonably modern systems	Java 8

Long-term modern enterprise compatibility	Java 17
Modern LTS platform	Java 21
Latest LTS available before Java 26	Java 25
Latest Java features	Java 26
Java 1.x	Only for historical/legacy requirements

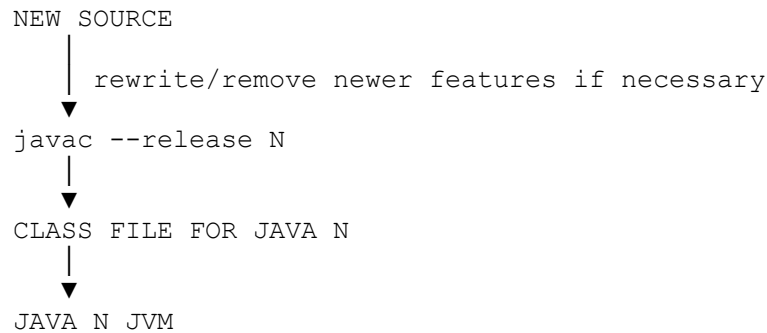
Java 8 is often an important compatibility floor, but it is **not automatically the correct target** for every application.

29. Final Summary

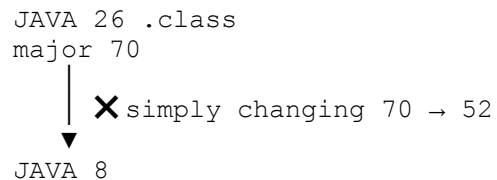
Question	Answer
Can Java 26 .class directly run on Java 8?	✗ No
Can Java 26 source be compiled for Java 8?	✓ Yes, if the source/API usage is Java 8-compatible
Command?	javac --release 8 MyApp.java
Java 8 class-file major?	52
Java 26 class-file major?	70
Can changing 70 to 52 downgrade a class?	✗ No
Can --release translate records to normal classes?	✗ No
Can --release translate var to explicit types?	✗ No

Can --release remove virtual threads?	✗ No
What does --release do?	Restricts source/API and generates bytecode for the selected Java release
Can Java 26 javac target every historical Java version?	✗ No
Can a Java 26 application be ported to older Java?	✓ Often, but increasingly difficult as the target becomes older
Can Java 26 source be ported to Java 1.4?	☐ Yes in principle, but requires substantial rewriting and an appropriate historical toolchain
Can Java 26 source be ported to Java 1.0?	☐ Only as a major historical port/rewrite, not ordinary modern compilation

The fundamental rule



Whereas:



does not work.

So if your goal is “**Java 26 → Java 1, 1.1, 1.2, ... 25**”, the technically correct approach is **source-level backporting plus compilation with an appropriate target toolchain**, not class-file-number manipulation.